

Assignment 7

Q1

[0.4 points]

Consider a 2-PDA that is a pushdown automaton equipped with two stacks, that can decide to push a symbol on neither stack, on either of the two stacks or on both stacks. Show how such a 2-PDA can recognise the language $A = \{a^k b^k c^k \mid k \geq 0\}$.

A1

The following 2-PDA will recognise language A:

- Push all a's that appear in front of the input tape to stack 1.
- Then push all b's that follow the a's in the input stream into stack 2. If it sees any a's at this stage, reject.
- When it sees the first c, pop stack 1 and stack 2 at the same time. After this, reject the input if it sees any a's (or) b's in the input stream.
- Pop stack1 and stack2 for each input character c it reads. If the input ends when both stacks are empty, accept. Otherwise reject.

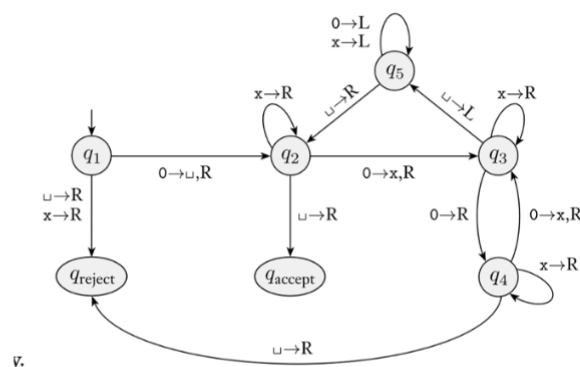
Therefore, B is recognized by 2-PDA.

Thus 2PDAs are more powerful then 1-PDAs.

Q2

[0.2 points]

Consider the TM M seen at lecture that decides the language $B = \{0^{2^n} \mid n \geq 0\}$ with state diagram:



Give the sequence of configurations that the M enters when started on the following input strings:

- 0
- 00
- 000
- 000000

A2

a)

$q_1 0$	(At $q_1, 0 \rightarrow \sqcup$, R goes to the q_2)
$\sqcup q_2 \sqcup$	(At $q_2, \sqcup \rightarrow$ R goes to the accept state)
$\sqcup \sqcup q_{\text{accept}}$	

b)

$q_1 00$	(At $q_1, 0 \rightarrow \sqcup$, R goes to the q_2)
$\sqcup q_2 0$	(At $q_2, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x q_3 \sqcup$	(At $q_3, \sqcup \rightarrow L$ goes to the q_5)
$\sqcup q_5 x \sqcup$	(At $q_5, x \rightarrow L$ goes to the q_5)
$q_5 \sqcup x \sqcup$	(At $q_5, \sqcup \rightarrow R$ goes to the q_2)
$\sqcup q_2 x \sqcup$	(At $q_2, x \rightarrow R$ goes to the accept state)
$\sqcup x q_2 \sqcup$	(At $q_2, \sqcup \rightarrow R$ goes to the accept state)
$\sqcup x \sqcup q_{\text{accept}}$	

c)

$q_1 000$	(At $q_1, 0 \rightarrow \sqcup$, R goes to the q_2)
$\sqcup q_2 00$	(At $q_2, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x q_3 0$	(At $q_3, 0 \rightarrow R$ goes to the q_4)
$\sqcup x 0 q_4 \sqcup$	(At $q_4, \sqcup \rightarrow R$ goes to the q_{reject})
$\sqcup x 0 \sqcup q_{\text{reject}}$	

d)

$q_1 000000$	(At $q_1, 0 \rightarrow \sqcup$, R goes to the q_2)
$\sqcup q_2 00000$	(At $q_2, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x q_3 0000$	(At $q_3, 0 \rightarrow R$ goes to the q_4)
$\sqcup x 0 q_4 000$	(At $q_4, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x 0 x q_3 00$	(At $q_3, 0 \rightarrow R$ goes to the q_4)
$\sqcup x 0 x 0 q_4 0$	(At $q_4, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x 0 x 0 x q_3 \sqcup$	(At $q_3, \sqcup \rightarrow L$ goes to the q_5)
$\sqcup x 0 x 0 q_5 x \sqcup$	(At $q_5, x \rightarrow L$ goes to the q_5)
$\sqcup x 0 x q_5 0 x \sqcup$	(At $q_5, 0 \rightarrow L$ goes to the q_5)
$\sqcup x 0 q_5 x 0 x \sqcup$	(At $q_5, x \rightarrow L$ goes to the q_5)
$\sqcup x q_5 0 x 0 x \sqcup$	(At $q_5, 0 \rightarrow L$ goes to the q_5)
$\sqcup q_5 x 0 x 0 x \sqcup$	(At $q_5, x \rightarrow L$ goes to the q_5)
$q_5 \sqcup x 0 x 0 x \sqcup$	(At $q_5, \sqcup \rightarrow R$ goes to the q_2)
$\sqcup q_2 x 0 x 0 x \sqcup$	(At $q_2, x \rightarrow R$ goes to the q_2)
$\sqcup x q_2 0 x 0 x \sqcup$	(At $q_2, 0 \rightarrow x$, R goes to the q_3)
$\sqcup x x q_3 x 0 x \sqcup$	(At $q_3, x \rightarrow R$ goes to the q_3)
$\sqcup x x x q_3 0 x \sqcup$	(At $q_3, 0 \rightarrow R$ goes to the q_4)
$\sqcup x x x 0 q_4 x \sqcup$	(At $q_4, x \rightarrow R$ goes to the q_4)
$\sqcup x x x 0 x q_4 \sqcup$	(At $q_4, \sqcup \rightarrow R$ goes to the q_{reject})
$\sqcup x x x 0 x \sqcup q_{\text{reject}}$	

Q3

[0.2 points]

Examine the formal definition of a Turing machine to answer the following questions, and explain your reasoning.

- Can a Turing machine ever write the blank symbol $\sqcup$ on its tape?
- Can the tape alphabet Γ be the same as the input alphabet Σ ?
- Can a Turing machine's head *ever* be in the same location in two successive steps?
- Can a Turing machine contain just a single state?

A3

- Yes. The tape alphabet Γ contains $\sqcup$. A Turing machine can write any characters in Γ on its tape.
- No. Σ never contains $\sqcup$, but Γ always contains $\sqcup$. So they cannot be equal.
- Yes. If the Turing machine attempts to move its head off the left-hand end of the tape, it remains on the same tape cell.
- No. Any Turing machine must contain two distinct states: q_{accept} and q_{reject} . So, a Turing machine contains at least two states.

Q4

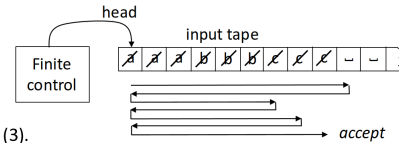
[0.4 points]

Design the state diagram of the Turing Machine discussed at lecture that recognises the language $B = \{a^k b^k c^k \mid k \geq 0\}$

TM – example

TM recognizing $B = \{a^k b^k c^k \mid k \geq 0\}$

- 1) Scan right until $\sqcup$ while checking if input is in $a^*b^*c^*$, *reject* if not.
- 2) Return head to left end.
- 3) Scan right, crossing off single a, b, and c.
- 4) If the last one of each symbol, *accept*.
- 5) If the last one of some symbol but not others, *reject*.
- 6) If all symbols remain, return to left end and repeat from (3).



Check-in 7.2

How do we get the effect of “crossing off” with a Turing machine?

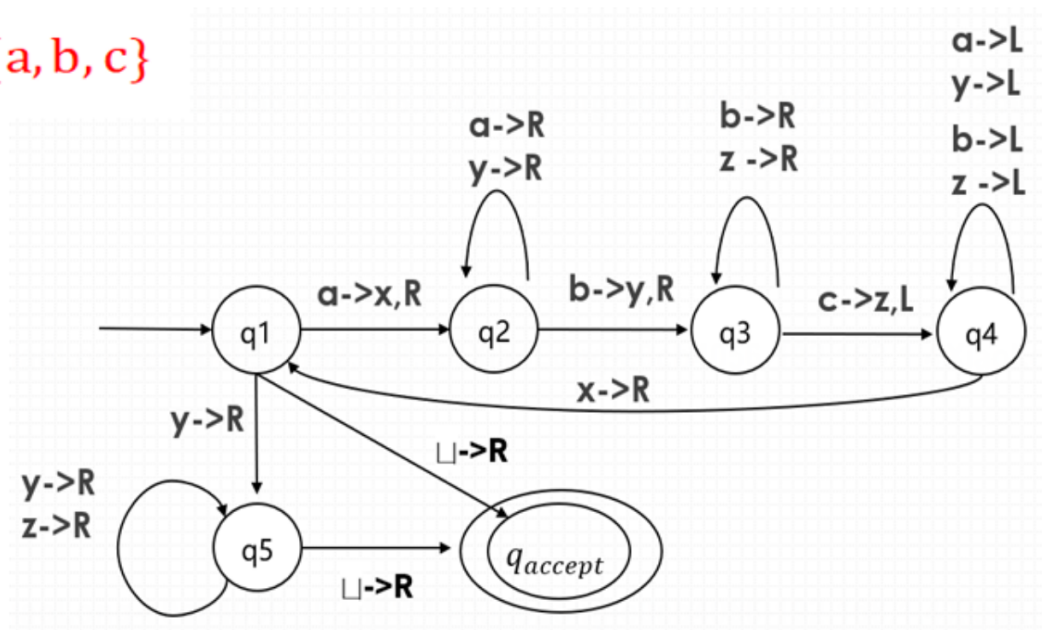
- a) We add that feature to the model.
- b) We use a tape alphabet $\Gamma = \{a, b, c, \cancel{a}, \cancel{b}, \cancel{c}\}$.
- c) All Turing machines come with an eraser.

6

Check-in 7.2

A4

$\Sigma = \{a, b, c\}$



Q5

[0.3 points]

Give *implementation-level* descriptions of Turing machines that decide the following languages over the alphabet $\{0,1\}$.

- a. $\{w \mid w \text{ contains an equal number of 0s and 1s}\}$
- b. $\{w \mid w \text{ contains twice as many 0s as 1s}\}$
- c. $\{w \mid w \text{ does not contain twice as many 0s as 1s}\}$

A5

The alphabet is $\{0,1\}$

$M_a =$ "On the input string ω :

- 1) Scan the input from left to right until blank crossing off the first 0. If there is no 0, scan the input from left to right to see if there is a 1. Reject if there is, Accept if there isn't.
- 2) Scan the input from left to right until blank crossing off the first 1. if there is no 1, reject.
- 3) Repeat from step 1."

$M_b =$ "On the input string ω :

- 1) Start from the left end of the input and scan right crossing off the first 0. If there is no 0, go to step 4.
- 2) Scan right crossing off the next 0. If there is no 0, go to step 4.
- 3) Move the head to the left end and Scan right crossing off the first 1. Reject if there is no 1. Repeat from step 1.
- 4) Scan from left to right until the first 1 or blank, reject if the head is on 1, accept if it is on blank.

$M_c =$ "On the input string ω :

- 1) Start from the left end of the input and scan right crossing off the first 0. If there is no 0, go to step 4.
- 2) Scan right crossing off the next 0. If there is no 0, go to step 4.
- 3) Move the head to the left end and Scan right crossing off the first 1. Accept if there is no 1. Repeat from step 1.
- 4) Scan from left to right until the first 1 or blank, accept if the head is on 1, reject if it is on blank.

Q6

A *Turing machine with left reset* is similar to an ordinary Turing machine, but the transition function has the form

$$\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\mathbf{R}, \text{RESET}\}.$$

If $\delta(q, a) = (r, b, \text{RESET})$, when the machine is in state q reading an a , the machine's head jumps to the left-hand end of the tape after it writes b on the tape and enters state r . Note that these machines do not have the usual ability to move the head one symbol left. Show that Turing machines with left reset recognize the class of Turing-recognizable languages.

A6

For a left reset machine to simulate an ordinary machine M , the machine can replace all of M 's left-moving states with the following algorithm for moving left over some character in the middle of the tape. Suppose the head wants to move left from the spot it is currently on.

1. Dot the current spot. Reset to the first spot.
2. Dot the first spot. Reset.
3. Move right until you hit a dot. Move right again.

4. (Case1) If the cell you are over already has a dot, it must be the original cell. Erase the dot, reset, and move right until you hit a dot. Now you are one spot to the left of your original spot, and you are done.
5. (Case2) If the cell you are over does not have a dot, dot it, reset, move until you hit a dot, erase the dot, reset, go to step 3.

Q7

A *Turing machine with stay put instead of left* is similar to an ordinary Turing machine, but the transition function has the form

$$\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{R, S\}.$$

At each point, the machine can move its head right or let it stay in the same position. Show that this Turing machine variant is *not* equivalent to the usual version. What class of languages do these machines recognize?

A7

We show that the languages recognised by this TM are regular languages.

Suppose the TM is $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$.

We now construct an NFA $(Q', \Sigma', \delta', q_{start}, F)$ as follows:

$$Q' = Q \times \Gamma \cup \{q_{start}, q_a, q_r\}$$

$$\Sigma' = \Gamma$$

$$F = \{q_a\}$$

Now we construct δ' :

1. Add rules $\delta'(q_{start}, a) = (q_0, a)$, for any $a \in \Gamma$.
2. If $\delta(q, a) = (r, b, R)$ and $r \notin \{q_{acc}, q_{rej}\}$, then add rules $\delta'((q, a), w) = (r, w)$, for any $w \in \Gamma$.
3. If $\delta(q, a) = (r, b, S)$ and $r \notin \{q_{acc}, q_{rej}\}$, then add rules $\delta'((q, a), \varepsilon) = (r, b)$.
4. If $\delta(q, a) = (q_{acc}, b, S/R)$, then add rules $\delta'((q, a), \varepsilon) = q_a$.
5. If $\delta(q, a) = (q_{rej}, b, S/R)$, then add rules $\delta'((q, a), \varepsilon) = q_r$.
6. Add rules $\delta'(q_a, w) = q_a, \delta'(q_r, w) = q_r$, for any $w \in \Gamma$.

The state pair (q, a) indicates that in a configuration, the state is q and the head is pointing to a . We can see that this NFA can recognise exactly the same language that are recognised by M . Thus the languages recognised by this TM variant are regular languages.

Q8

[0.25 points]

Show that the collection of decidable languages is closed under the operation of:

- | | |
|-------------------|---------------------|
| a. union. | d. complementation. |
| b. concatenation. | e. intersection. |
| c. star. | |

A8

a)

Let L_1 and L_2 be two decidable languages. M_1 and M_2 be the Turing machines that decide L_1 and L_2 respectively.

There exists a Turing machine M' such that it decides $L_1 \cup L_2$, i.e.

$$L(M') = L_1 \cup L_2.$$

The description of M' is as follows:

M' = on input w :

1. Run M_1 on w . If M_1 accepts, then accept.
2. Else run M_2 on w . If M_2 accepts, then accept.
3. Else reject.

M' accepts w if either M_1 or M_2 accepts it. If both reject, M' rejects.

Therefore,

$$L(M') = L_1 \cup L_2.$$

The decidable languages are closed under union.

b)

Let L_1 and L_2 be two decidable languages. M_1 and M_2 be the Turing machines that decide L_1 and L_2 respectively.

There is a Turing machine M' such that, it decides concatenation of L_1 and L_2 i.e.,

$$L(M') = L_1 \circ L_2.$$

The description of M' is as follows:

M' = on input w :

1. Split w into two parts w_1, w_2 such that $w = w_1w_2$.
2. Run M_1 on w_1 . If M_1 rejected then reject.
3. Else run M_2 on w_2 . If M_2 rejected then reject.
4. Else accept.

Try each possible cut of w . If first part is accepted by M_1 and the second part is accepted by M_2 then w is accepted by M' . Else, w does not belong to the concatenation of languages and is rejected.

Therefore,

$$L(M') = L_1 \circ L_2.$$

The decidable languages are closed under concatenation.

c)

Suppose A is a decidable languages. Turing machine M_1 decides A and Turing machine. Assume B is the star of A . To demonstrate B is a decidable language, we only need to construct a Turing machine M_2 that decides B .

Here's the description:

$M_2 =$ "on the input ω

1. Split ω into any numbers of strings in all possible ways that there can be no more than one string that is empty (finite).
2. Run M_1 on every string.
3. If in one splitting way, all of the strings are accepted, accept. Otherwise, reject."

d)

For a Turing decidable language L , Turing machine decides language M then the complement is M' on input w .

The description of M' is as follows:

$M' =$ on input w :

1. Accepts if M rejects
2. Else accept.

Since M' does the opposite of what ever M does, it decides the complement of L .

Therefore, decidable languages are closed under complementation.

e)

Let L_1 and L_2 be two Turing decidable languages. M_1 and M_2 be the Turing machines that decide L_1 and L_2 respectively.

There is a Turing machine M' such that it decides the intersection of L_1 and L_2 , i.e.,

$$L(M') = L_1 \cap L_2.$$

The description of M' is as follows:

$M' =$ on input w :

1. Run M_1 on w . If M_1 rejects, then reject.
2. Else run M_2 on w . If M_2 rejects, then reject.
3. Else accept.

M' accepts w if both M_1 and M_2 accept it. If either of them rejects, then M' rejects w .

Therefore,

$$L(M') = L_1 \cap L_2.$$

The decidable languages are closed under intersection.

Q9

[0.15 points]

Show that the collection of Turing-recognisable languages is closed under the operation of:

- a. union.
- b. concatenation.
- c. star.

A9

Let L_1 and L_2 be two Turing-recognizable languages, and let M_1 and M_2 be the Turing machines that recognize L_1 and L_2 respectively.

a)

For a non-deterministic Turing machine M that recognizes the language $L_1 \cup L_2$:

On input string s ,

- Put s on two tapes repeatedly;
- Run M_1 and M_2 on different tapes for one step alternately;
- If either of the machine accepts, accept; otherwise, reject.

So that the collection of Turing-recognizable languages is closed under the operation of union.

b)

For a non-deterministic Turing machine M that recognizes the language $L_1 L_2$:

On input string s ,

- Divide s into a and b non-deterministically;
- For every dividing method, do the following recurrently,
 - If M_1 on a is already accepted, run M_2 on b for one step; else run M_1 on a for one step, if M_1, M_2 are all accepted, accept.
 - If either M_1 or M_2 is halt and rejected, stop trying this dividing method.
- If all possible dividing methods are stopped from trying, reject.

So that the collection of Turing-recognizable languages is closed under the operation of concatenation.

c)

For a non-deterministic Turing machine M that recognizes the language L_1^* :

On input string s ,

- Divide s into $w_1, w_2 \dots w_n$ non-deterministically, while there is no more than 1 sub-string that is empty.
- For every dividing method, do the following recurrently,
 - Run M on every unaccepted sub-string for one step; if M on every sub-string is accepted, accept.
 - If M on all sub-string is halt and rejected, stop trying this dividing method.

- If all possible dividing methods are stopped from trying, reject.

So that the collection of Turing-recognizable languages is closed under the operation of star.

Q10

[0.3 points]

Give Turing machine programs for the following operations:

- Given a positive binary number x return $x - 1$
- Given two binary numbers x and $y, x > y$ return $x - y$
- Given two binary numbers x and $y, x > y$ return x/y

A10

a)

Suppose M is the corresponding turning machine. Here is the turning machine algorithms: "First shift the head to the right-hand end of the tape. Then sweep right to left across the tape and change 0 into 1 until we see the first 1. Once we see the first 1, we change it into 0 and our machine halts."

b)

Assume that the input tape is like,

$$x\#y \sqcup \sqcup \sqcup \dots$$

We can use the following Turing machine to give the result.

$M =$ On input string ω

- Shift right till hit the $\sqcup$.
- Move towards left.

Before encounters $\#$, we discuss the situation.

- When encounters 0, replace it with 1 and continue moving.
- When encounters 1, replace it with 0 and just shift left until it hits the $\#$.

- After hitting $\#$, still move towards left.

- When encounters 0, replace it with 1 and continue moving.
- When encounters 1, replace it with 0 and just shift left until it hits the $\sqcup$.

- After it hits $\sqcup$, shift right and check if there is a 1 after encountering $\#$. If it does, shift right till hit the $\sqcup$, go back to Step 2. Otherwise, just return and the value left before $\#$ is the desired value.

c)

We use 3-tapes Turing machine in this problem (Note that a multi-tape Turing machine is equivalent to a single-tape turning machine).

Suppose M is the corresponding turning machine. Here is the turning machine algorithms:

Firstly, we write $x\#y$ on the first tape and we copy $x\#y$ to the second tape. We write 0 on the third tape.

Secondly, we calculate $x - y$ on the second tape (The details of $x - y$ is shown in problem (b)). If $x > y$, we copy y on the first tape to the second tape and add 1 to the third tape. We should keep doing the second step until $x < y$.

Notice:

1. As shown in problem b, each time we calculate $x - 1$ and $y - 1$. If x is 0 and y isn't 0, $x < y$.
2. How to add x and 1 is shown in the class and the algorithm is "Start from the end of the string going left, while we see a 1 write a 0. Once we see a 0 write a 1. If no first 0, shift everything to the right and add a 1 at the beginning".

If $x < y$, we return the results, where the number on the third tape is the quotient and the x on the second tape is the remainder.